

FundFinder — Architecture

CSR-fit research tool: given a company name, it scrapes public sources, scores CSR/partnership fit with an LLM, and outputs a web report + Word/Excel deep-dive.

1. Stack

Layer	Tech
Web framework	FastAPI + Jinja2 + HTMX (server-rendered, polling-based)
Auth	Supabase Auth (Google OAuth), itsdangerous signed cookie sessions
DB / Storage	Supabase (Postgres + Storage bucket <code>screening-files</code>)
LLM	Anthropic (<code>app/pipeline/llm.py</code>)
Search	Google Custom Search API (<code>google_search.py</code>)
Scraping	<code>requests</code> + <code>retry/backoff</code> , <code>app/pipeline/scrapper.py</code>
Reports	<code>python-docx</code> (Word), <code>openpyxl</code> (Excel)
Background jobs	In-memory dict (<code>JOBS</code>) + <code>asyncio.create_task</code> , no external queue

2. Request Flow

None

User submits company name (POST /screen)

|

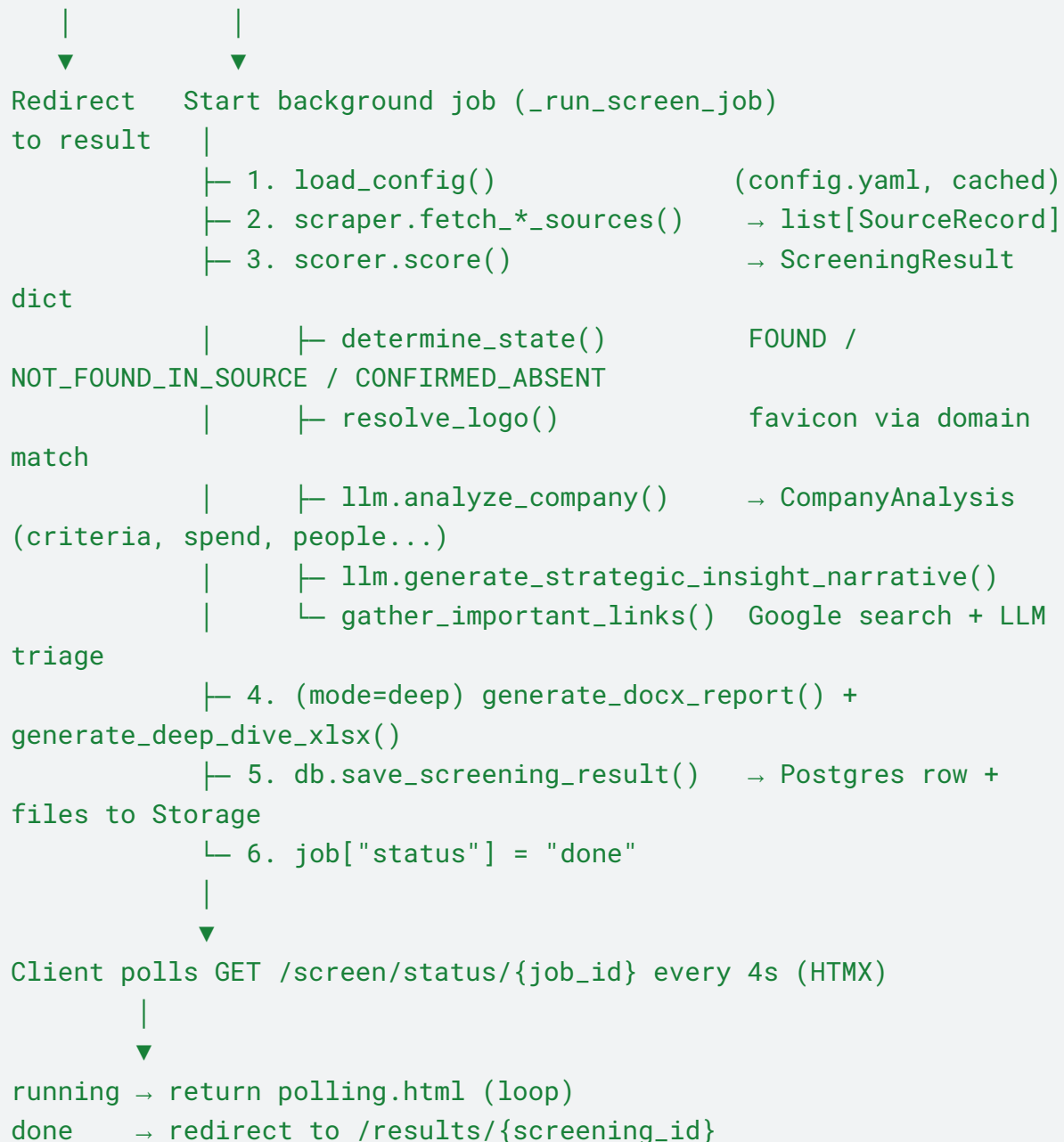


Reuse check: is there a fresh screening for this company+mode already?

|

(screen: <6h old, deep: <3d old)

yes — no



Job states: running → done|error → delivered|delivered_error (short TTL cleanup via `_prune_expired_jobs`). In-flight dedup keyed by (company_slug, mode) prevents duplicate concurrent runs.

3. Source Gathering (`scraper.py`, `search_budget.py`)

Source name	What it fetches	Priority weight
<code>india_csr_page</code>	Company's own CSR/sustainability page	1.35
<code>annual_report / global_annual_report</code>	PDF annual/sustainability report	1.3
<code>mca_portal</code>	MCA (Ministry of Corp Affairs) CSR-2 filing	1.25
<code>national_csr_portal</code>	csr.gov.in	1.2
<code>mca_via_search</code>	MCA data via search proxy	1.05
<code>partner_search</code>	NGO/implementation partner mentions	1.0
<code>plans_search</code>	Announced plans / RFPs	1.0
<code>people_search</code>	LinkedIn CSR decision-makers	0.95
<code>sector_eligibility_search</code>	Sector & Section 135 mandate signals	0.75

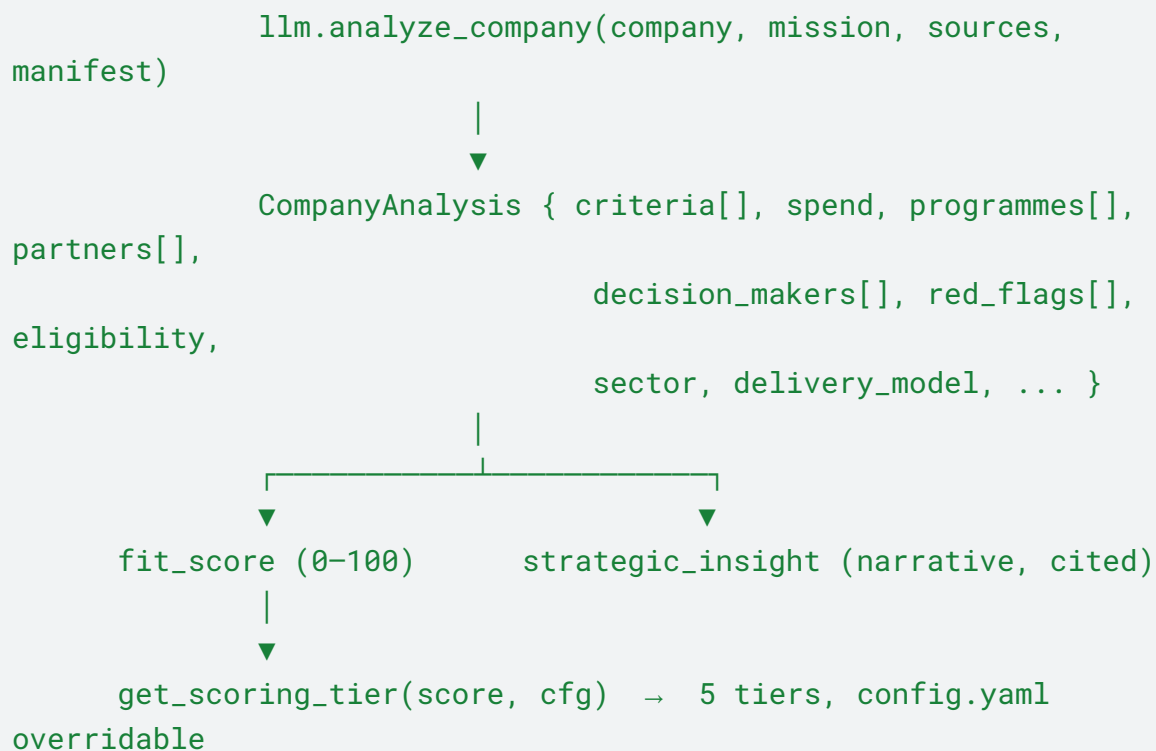
`SearchBudget` caps Google/DDGS queries per run, reserving a floor for high-value categories (`partner_search`, `people_search`, `education_programme_search`) so cheap early lookups can't starve them. Every found source registers into `SourceRegistry`, which assigns a stable `[N]` citation number reused across the LLM analysis, DOCX, XLSX, and HTML report.

4. Scoring Pipeline (`scorer.py`)

None

```
sources → structure_all_sources() →
build_token_budgeted_evidence()
          (textproc.py: sentence split, relevance scoring,
dedup)
```





Tiers (`TIER_DEFAULT`):

Min score	Label	Action
90	Immediate Target	CEO-to-CEO outreach in 7 days
80	Strong Fit	Full pitch, discovery call
65	Conditional	Strengthen evidence, find intro path
45	Watchlist	Monitor quarterly
0	Not a Target	Deprioritise

If no evidence found and state is `CONFIRMED_ABSENT`, or if the LLM call fails, scoring short-circuits to an **UNSCORED** result (`TIER_UNSCORED`) rather than guessing — no wasted Anthropic calls.

5. Data Model

Table

Key fields

<code>screenings</code>	company, company_slug, mode, state, fit_score, tier_*, analysis (jsonb), sources (jsonb), source_bank (jsonb), decision_makers, important_links, logo_url, user_id, created_at
<code>screening_files</code>	screening_id → storage_path (docx/xlsx in <code>screening-files</code> bucket), signed URLs generated on read
<code>job_events</code> / <code>job_runs</code>	append-only audit log per screening_id / per company+mode run (start, sources_fetched, scored, done, error)
<code>company_notes</code>	free-text notes per company_slug

`company_slug` (alnum-normalized) is the de-dup/history key across all tables.

6. Output Formats

Format	Trigger	Generator	Contents
HTML report	always	<code>reporters.py</code> + <code>results.html</code>	Score, tier, criteria bars, spend, programmes, partners, decision-makers, sources, downloads
<code>.docx</code>	mode=deep	<code>docx_reporter.py</code>	Branded exec summary, criteria table, red flags, spend history, governance signals
<code>.xlsx</code>	mode=deep	<code>xlsx_reporter.py</code>	9 tabs: Verdict, Criteria, Financials, Programmes, Decision-Makers, Governance, Red Flags, Approach, Sources

Both files are saved to Supabase Storage and also base64-embedded as a fallback if storage is unavailable.

7. Auth

Google OAuth via Supabase (`auth.py`). Session = signed cookie (`itsdangerous`, 30-day max age), not a Supabase JWT directly — payload stores `user_id`, `email`, `access_token`, `refresh_token`. If `supabase_auth_configured` is false, auth is fully bypassed (dev mode).

8. Config

`config.yaml` (hot-reloadable via `reload_config()`) supplies: `org_mission`, `decision_tiers_v7`, `score_bands`, `search_source_toggles`. Env vars (`.env` / `Settings`) supply API keys, DB URL, session secret, model name, quota caps.

9. Example Run

None

```
POST /screen company="Tata Consultancy Services" mode=deep
  → job_id=abc123, poll every 4s
  → sources: india_csr_page ✓, annual_report ✓, mca_portal ✗,
people_search ✓ (3 hits)
  → analysis: delivery_model=HYBRID, fit_score=82, tier=Strong Fit
  → docx: tap_csr_tata_consultancy_services_deep.docx
  → xlsx: tap_csr_tata_consultancy_services_deep_dive.xlsx
  → GET /results/{screening_id} → HTML report with [1]..[12] cited
sources
```

10. Key Design Choices

- **No task queue** — jobs live in a process-local dict; fine for single-instance deploys, won't survive a restart or scale horizontally without moving to Redis/Celery.
- **Citation integrity** — `SourceRegistry` + `strip_unknown_citation_tokens` prevents the LLM from hallucinating a `[N]` that doesn't map to a real fetched source.
- **Graceful degradation** — every external dependency (Supabase, Google Search, Anthropic) has a "not configured" no-op path; the app still runs in reduced mode.
- **Freshness cache** — re-screening the same company within a TTL window redirects to the cached result instead of re-running the pipeline (saves LLM/search spend).